

Лабораторная работа №3 “Реализация формирования линейного кода в терминах набора инструкций абстрактной виртуальной машины”

Цель работы

Реализовать формирование линейного кода в терминах набора инструкций абстрактной виртуальной машины для набора подпрограмм путем анализа графа потока управления.

Описание реализации

Структуры данных

Реализованы следующие структуры данных для представления информации об элементах образа программы:

1. **VMRegister** - перечисление типов регистров (R0-R7, IP, SP, BP, FLAGS)
2. **Operand** - структура операнда с поддержкой типов: регистр, немедленное значение, память, метка
3. **VMInstruction** - структура инструкции с массивом операндов
4. **VMDataItem** - структура элемента данных
5. **VMLabel** - структура метки с именем и адресом
6. **VMProgram** - структура образа программы (инструкции, данные, метки)

Специализированные структуры для генерации

1. **RegisterAllocator** - отслеживает использование регистров R0-R7
2. **LocalVarMapping** - связывает имя локальной переменной с её офсетом и регистром
3. **ArgumentMapping** - связывает имя аргумента с его индексом и регистром
4. **FunctionContext** - контекст генерации кода для одной функции

Алгоритм построения образа программы

Основной алгоритм реализован в функции `generate_function_code`:

1. **Генерация пролога функции:** PUSH BP, MOV BP, SP, SUB SP для локальных переменных
2. **Обход базовых блоков** в топологическом порядке:
 - Для каждого блока добавляется метка вида `func_block_label`
 - Если блок содержит операции типа “block”, генерируются все инструкции блока

— Проверяется, содержит ли блок управляющие конструкции (if/while)

3. Генерация successors:

- Для обычных и пустых блоков генерируются JMP к successor-ам
- Для блоков с if/while successors НЕ генерируются (они сами управляют переходами)

4. Топологическая сортировка блоков обеспечивает правильный порядок генерации

Особенности реализации управления потоком

Генерация successors реализована с умным определением типов блоков:

```
static bool is_control_flow_statement(const char *type) {  
    return (strcmp(type, "ifStmt") == 0 ||  
            strcmp(type, "if") == 0 ||  
            strcmp(type, "whileStmt") == 0 ||  
            strcmp(type, "while") == 0);  
}
```

- Обычные блоки (присваивания, varDecl) ☐ генерируют JMP к successors
- Блоки с if/while ☐ НЕ генерируют successors (управление внутри generate_statement)
- Пустые блоки ☐ генерируют JMP к successors для связывания блоков

Генерация условий для операторов сравнения:

Для выражений вида `a op b` генерируется оптимальный код без двойного CMP:

Оператор	Логика прыжка	Инструкции
<code>a > b</code>	Прыгать в else если <code>a ☐ b</code>	JL else, JE else
<code>a < b</code>	Прыгать в else если <code>a ☐ b</code>	JG else, JE else
<code>a == b</code>	Прыгать в else если <code>a $\neq$ b</code>	JNE else

Результаты тестирования

Тест 1: Условный оператор `c >`

Исходный код:

```
int test_ge(int a) {  
    int result;  
    if (a > 5) {  
        result = 100;  
    }
```

```

    } else {
        result = 0;
    }
}

```

Сгенерированный код:

```

test_ge:
    PUSH BP
    MOV BP, SP
    SUB SP, 4
test_ge_stmt_2:
    MOV R0, R2          ; a = R2 (аргумент)
    MOV R1, 5           ; константа 5
    CMP R0, R1          ; сравнение a с 5
    JL else_0           ; прыгнуть если a < 5
    JE else_0           ; прыгнуть если a == 5
    MOV R0, 100         ; тело if (a > 5)
    JMP if_end_0
else_0:
    MOV R0, 0           ; тело else (a ≤ 5)

```

Тест 2: Цикл while

Исходный код:

```

int count_to_3() {
    int i;
    i = 0;
    while (i < 3) {
        i = i + 1;
    }
}

```

Сгенерированный код:

```

count_to_3:
    PUSH BP
    MOV BP, SP
    SUB SP, 4
count_to_3_stmt_2:
    MOV R0, 0          ; i = 0
while_start_0:

```

```

MOV R1, R0
MOV R2, 3
CMP R1, R2
JG while_end_0      ; выход если i > 3
JE while_end_0      ; выход если i = 3
MOV R1, R0          ; тело: i = i + 1
MOV R2, 1
ADD R1, R2
MOV R0, R1
JMP while_start_0   ; переход в начало цикла

```

Тест 3: Вложенные конструкции

Исходный код:

```

int simple_test() {
    int x;
    int result;
    x = 0;
    result = 0;
    while (x < 3) {
        if (x > 0) {
            result = result + 1;
        }
        x = x + 1;
    }
}

```

Сгенерированный код (основная часть):

```

while_start_0:
    MOV R2, R0      ; загрузка x
    MOV R3, 3       ; константа 3
    CMP R2, R3
    JG while_end_0  ; выход при x ≥ 3
    JE while_end_0
    ; if (x > 0)
    MOV R2, R0
    MOV R3, 0
    CMP R2, R3
    JL if_end_0     ; пропустить if при x ≤ 0
    JE if_end_0

```

```

        ; тело if: result++
MOV R2, R1
MOV R3, 1
ADD R2, R3
MOV R1, R2
if_end_0:
        ; x++
MOV R2, R0
MOV R3, 1
ADD R2, R3
MOV R0, R2
JMP while_start_0

```

Выводы

Разработанный модуль успешно генерирует линейный код из графов потока управления. Ключевые достижения:

1. **Правильная генерация successors:** обычные блоки связываются JMP, управляющие конструкции не создают лишних переходов
2. **Оптимизация условий:** генерация прямых переходов без двойного сравнения для операторов сравнения
3. **Поддержка вложенных конструкций:** корректная работа со сложными случаями (циклы с условиями внутри)
4. **Работа с CFG:** поддерживаются как типы из AST (ifStmt, whileStmt), так и из CFG (if, while)

Модуль готов к использованию и может быть расширен дополнительными инструкциями ВМ и оптимизациями.